

Evoluzione del progetto e scelte implementative

Scelta del tema ed evoluzione dell'idea iniziale

All'inizio del progetto volevamo lavorare su un problema legato al **Frequent Itemset Mining**, perché ci sembrava molto vicino agli argomenti visti a lezione, in particolare al paradigma **MapReduce**.

L'idea iniziale era abbastanza naturale: partire da un grande insieme di transazioni, distribuire i dati tra più nodi, contare localmente gli item e i pattern frequenti, e poi aggregare i risultati. Per questo motivo abbiamo iniziato a cercare articoli su Google Scholar riguardanti il mining di item frequenti, concentrandoci in particolare su algoritmi come **FP-Growth** e **Parallel FP-Growth**.

Il **Frequent Itemset Mining** ha come obiettivo quello di trovare insiemi di item che compaiono spesso insieme all'interno delle transazioni. Un esempio classico è la market basket analysis: se molti clienti acquistano frequentemente insieme gli item A e B, allora il pattern $\{A, B\}$ può essere considerato frequente.

In questa formulazione classica, però, gli item sono trattati in modo binario:

- l'item è presente nella transazione;
- oppure l'item non è presente.

Non viene invece considerata la quantità acquistata. Per esempio, dal punto di vista del Frequent Itemset Mining classico, due transazioni come:

A, B

A, B

sono equivalenti, anche se nella realtà potrebbero corrispondere a quantità molto diverse, ad esempio:

A: 1, B: 2

A: 6, B: 12

Durante il confronto con il professore è emersa la richiesta di modificare l'idea iniziale, aggiungendo anche l'informazione sulle **quantità** associate agli item. Questo ci ha fatto capire che il problema non era più semplicemente un Frequent Itemset Mining classico.

A questo punto è stato necessario chiarire meglio la differenza tra le diverse tipologie di mining. Nel Frequent Itemset Mining classico ciò che conta è soltanto il numero di transazioni che contengono un certo insieme di item. Con l'introduzione delle quantità, invece, il problema cambia in modo significativo: non si sta più cercando solo quali item compaiono spesso insieme, ma anche quali combinazioni di item e quantità sono frequenti.

Questo nuovo problema rientra nell'ambito del **Quantitative Itemset Mining**, dove non si considera più soltanto la presenza o assenza degli item, ma anche valori numerici associati a essi, come le quantità acquistate.

Durante un confronto successivo abbiamo però capito che affrontare il Quantitative Itemset Mining in modo generale sarebbe stato troppo ampio rispetto agli obiettivi del progetto. Il problema non era più soltanto trovare pattern frequenti, ma definire anche quale tipo di relazione quantitativa fosse interessante cercare.

Per questo motivo abbiamo proposto di concentrarci su una forma più specifica di relazione quantitativa: la **proporzionalità** tra gli item di un pattern. L'idea era riconoscere pattern che, pur avendo quantità assolute diverse, rappresentano la stessa struttura relativa.

Per esempio, i due pattern:

A: 1, B: 2
A: 6, B: 12

sono diversi dal punto di vista delle quantità assolute, ma condividono la stessa proporzione:

A : B = 1 : 2

Da qui è nata l'idea di introdurre una fase di analisi proporzionale, basata sulla riduzione delle quantità tramite massimo comune divisore. In questo modo ogni pattern quantitativo può essere separato in due parti:

- una **forma canonica**, che rappresenta la proporzione tra gli item;
- un **fattore di scala**, che rappresenta la grandezza assoluta del pattern.

Quindi, mentre il Frequent Itemset Mining classico avrebbe semplicemente riconosciuto che A e B compaiono insieme, il nostro obiettivo è diventato più specifico: riconoscere quando pattern quantitativi diversi rappresentano in realtà la stessa struttura proporzionale.

Dato che il focus del progetto era comunque legato a Spark e al calcolo distribuito, inizialmente siamo partiti cercando direttamente un algoritmo parallelo. Per questo motivo abbiamo individuato il paper su **Parallel FP-Growth**, che sembrava essere il riferimento più adatto per scalare il Frequent Itemset Mining su grandi dataset.

Tuttavia, una volta iniziata la lettura, ci siamo resi conto che partire direttamente dalla versione parallela rendeva difficile capire davvero il funzionamento dell'algoritmo. Il paper su PFP assume infatti una certa familiarità con FP-Growth e con concetti come FP-tree, F-list, conditional pattern base e conditional FP-tree. Senza aver chiaro prima l'algoritmo di base, molti passaggi della versione parallela risultavano poco comprensibili.

Per questo motivo siamo tornati indietro e abbiamo studiato prima il paper originale su **FP-Growth**. Questa fase è stata necessaria per capire come viene costruito l'FP-tree, come vengono ordinati gli item tramite la F-list, come si costruiscono i conditional tree e come vengono generati ricorsivamente i pattern frequenti.

Solo dopo aver chiarito questi passaggi è stato possibile tornare alla versione parallela e interpretare meglio il ruolo di PFP nel distribuire il carico computazionale.

A questo punto il progetto ha assunto una struttura più chiara:

1. usare FP-Growth/PFP come base per l'estrazione dei pattern frequenti;
2. adattare il dataset quantitativo trasformando le coppie (`item`, `quantity`) in token;
3. applicare successivamente una fase di post-processing per riconoscere e raggruppare i pattern proporzionalmente equivalenti.

Durante la fase iniziale abbiamo anche considerato altre possibili direzioni, come il **Graph Mining**. Tuttavia, questa strada è stata esclusa come obiettivo principale, perché gli algoritmi su grafi richiedono spesso modelli di calcolo specifici, come Pregel o approcci vertex-centric, e ci avrebbero portato lontano dal focus iniziale su MapReduce, Spark e FP-Growth.

In sintesi, l'evoluzione iniziale del progetto non è stata lineare. Siamo partiti dal Frequent Itemset Mining, siamo stati spostati verso il Quantitative Itemset Mining a causa dell'introduzione delle quantità, abbiamo cercato subito una soluzione parallela per rimanere coerenti con Spark, ma poi abbiamo dovuto fare un passo indietro per comprendere bene FP-Growth prima di poter implementare o adattare Parallel FP-Growth. Da questo percorso è nata infine l'idea di concentrarci su un'interpretazione del Quantitative Itemset Mining più specifica basata sulle relazioni proporzionali tra pattern.

Studio dell'algoritmo e sviluppo dell'implementazione

Dopo aver individuato FP-Growth e Parallel FP-Growth come riferimenti principali, ci siamo resi conto che la parte più difficile non era soltanto scrivere il codice, ma capire davvero il funzionamento dell'algoritmo.

Per questo motivo, prima di continuare con l'implementazione, io e il mio collega ci siamo ritrovati più volte a discutere il problema, a fare esempi a mano e a simulare la computazione dell'algoritmo anche su lavagna. Questo passaggio è stato utile soprattutto per chiarire alcuni punti che inizialmente non erano immediati, come la costruzione dell'FP-tree, il ruolo della F-list, la generazione dei conditional pattern base e la ricorsione sui conditional FP-tree.

In una prima fase, anche a causa di queste difficoltà implementative, ci siamo concentrati soprattutto sulla comprensione dell'algoritmo di base. Per un certo periodo abbiamo quindi perso parzialmente di vista l'aspetto parallelo del progetto e ci siamo ritrovati, di fatto, a implementare una versione locale di FP-Growth. Questo però non è stato tempo perso: ci ha permesso di capire meglio la logica interna dell'algoritmo e di costruire diverse funzioni che si sono poi rivelate utili anche nella versione parallela.

Successivamente siamo tornati sul paper di Parallel FP-Growth e abbiamo recuperato il collegamento con l'obiettivo iniziale del progetto, cioè l'implementazione distribuita in Spark. A partire dalla versione locale, abbiamo quindi riorganizzato il codice per adattarlo alla logica di PFP, introducendo la suddivisione in gruppi, la generazione delle group-dependent transactions e il mining locale sui diversi shard.

Inizialmente tutto il codice è stato sviluppato all'interno di un Jupyter Notebook. Questa scelta ci ha aiutato nella fase esplorativa, perché permetteva di separare il lavoro in sezioni, controllare gli output intermedi e capire meglio cosa stava succedendo in ogni passaggio dell'algoritmo. Una volta che la struttura generale è diventata più chiara, abbiamo però convertito il codice in file `.py`, in modo da renderlo più ordinato, modulare e più pratico da eseguire.

Parallelamente allo sviluppo della parte FP-Growth/PFP, abbiamo lavorato anche sulla fase di analisi proporzionale. Anche in questo caso l'idea finale non è nata subito in modo definitivo. Inizialmente abbiamo discusso diverse possibili strategie per confrontare pattern quantitativi e riconoscere relazioni tra quantità, inclusi approcci più euristici e greedy. Tuttavia, queste soluzioni risultavano meno pulite e più difficili da giustificare formalmente.

Alla fine siamo arrivati alla conclusione che il modo più semplice e robusto per rappresentare la proporzione interna di un pattern fosse utilizzare il **massimo comune divisore** delle quantità. In questo modo ogni pattern quantitativo può essere decomposto in una forma canonica, che rappresenta la proporzione tra gli item, e in un fattore di scala, che rappresenta la grandezza assoluta del pattern.

Per esempio, un pattern come:

```
A: 6, B: 12
```

può essere ridotto dividendo le quantità per il loro massimo comune divisore:

```
gcd(6, 12) = 6
```

ottenendo quindi:

```
A: 1, B: 2
```

In questo modo pattern quantitativamente diversi, ma proporzionalmente equivalenti, possono essere raggruppati insieme. Questa scelta ci ha permesso di collegare la parte di mining frequente con una successiva fase di compressione e interpretazione dei risultati.

Considerazioni finali sull'evoluzione del progetto

Guardando il percorso nel suo insieme, il progetto si è evoluto in modo abbastanza diverso rispetto all'idea iniziale. Inizialmente pensavamo di implementare un algoritmo parallelo per il Frequent Itemset Mining, ma l'introduzione delle quantità ha reso necessario ripensare il problema.

Questo ci ha portato prima verso il Quantitative Itemset Mining e poi verso una formulazione più specifica basata sulle relazioni proporzionali tra pattern. In questo senso, la parte proporzionale non è stata aggiunta solo come dettaglio finale, ma è diventata il modo con cui abbiamo cercato di dare un significato più interessante ai pattern quantitativi estratti.

Anche dal punto di vista implementativo il percorso non è stato immediato. In una prima fase ci siamo concentrati molto sulla comprensione di FP-Growth, arrivando quasi a implementare una versione locale dell'algoritmo. Successivamente abbiamo recuperato l'obiettivo iniziale legato al calcolo distribuito e abbiamo adattato il lavoro alla logica di Parallel FP-Growth e Spark.

Questa evoluzione ha messo in evidenza una difficoltà importante: non sempre passare da un algoritmo teorico a una versione distribuita è diretto. Prima è stato necessario capire bene la struttura dell'algoritmo sequenziale, poi individuare quali parti potevano essere riutilizzate e infine riorganizzarle in una pipeline compatibile con Spark.

Il risultato finale è quindi una pipeline composta da più fasi:

1. trasformazione delle transazioni quantitative in token;
2. estrazione dei pattern frequenti tramite una versione parallela di FP-Growth;
3. riconversione dei token nei corrispondenti item quantitativi;
4. normalizzazione dei pattern tramite massimo comune divisore;
5. raggruppamento dei pattern proporzionalmente equivalenti;
6. costruzione di una struttura finale basata sui fattori di scala.

In conclusione, il progetto non si è limitato a implementare un algoritmo già esistente, ma ha richiesto diversi passaggi di adattamento, correzione e reinterpretazione del problema. Gli errori e i cambi di direzione sono stati parte del processo, perché hanno permesso di chiarire meglio sia il funzionamento di FP-Growth/PFP, sia il significato della componente quantitativa che volevamo aggiungere.

Mattia Cappelletti, Matteo Baldi